Звіт з індивідуального завдання з дисципліни «Технології DevOps»

на тему: «Пропозиція з реалізації інфраструктури DevOps для сервера „Розумного будинку“ на базі апаратної платформи Raspberry Pi з підтримкою інтеграції цифрового двійника»

2026

Пропозиція з реалізації інфраструктури DevOps для сервера «Розумного будинку» на базі апаратної платформи Raspberry Pi з підтримкою інтеграції цифрового двійника

Зайкін Андрій Вікторович,
здобувач другого курсу магістратури, групи КІ-25-1м,
кафедри комп’ютерної інженерії та електроніки,
Кременчуцький національний університет імені Михайла Остроградського

Дружинін Валерій Олегович,
асистент кафедри комп’ютерної інженерії та електроніки,
Кременчуцький національний університет імені Михайла Остроградського

АНОТАЦІЯ

Робота продовжує індивідуальне завдання 1 і пропонує інфраструктуру DevOps, яка приводить сервер «Розумного будинку» на Raspberry Pi до сучасного стану ІТ-галузі: контейнеризація, оркестрування, гібридна хмара та кластеризація. Об’єктом дослідження є процес експлуатації та оновлення програмного забезпечення сервера, а предметом – архітектура та засоби інфраструктури для його обслуговування.

У роботі розглянуто клієнт-серверну модель системи, проаналізовано багаторівневу модель edge – fog – cloud, розраховано потребу в серверних ресурсах (2 повідомлення/с, ≈ 104 МБ сирих даних на добу), описано вміст семи контейнерів та алгоритм їх оркестрування в k3s, порівняно хмарні сервіси з безкоштовними тарифами для студентів і запропоновано кластер із трьох вузлів Raspberry Pi та хмарного вузла, що витримує відмову одного вузла.

Результати можуть бути використані для інших науково-технічних проєктів кафедри, побудованих навколо «Розумного будинку» та мікроконтролерів ESP8266.

Ключові слова: клієнт-сервер, edge computing, контейнер, Kubernetes, k3s, хмарний сервіс, кластер, Raspberry Pi, цифровий двійник.

ЗМІСТ

ВСТУП.....4
1 МОДЕЛЬ КЛІЄНТ-СЕРВЕР ТА ЇЇ ВПРОВАДЖЕННЯ У ПРОЄКТ.....5
1.1 Сутність і різновиди моделі клієнт-сервер.....5
1.2 Клієнт-серверна модель сервера «Розумного будинку».....5
2 АНАЛІЗ БАГАТОРІВНЕВОЇ МОДЕЛІ ІНФРАСТРУКТУРИ.....8
2.1 Варіанти багаторівневої моделі.....8
2.2 Переваги та недоліки для науково-технічного проєкту.....9
3 ВИЗНАЧЕННЯ НЕОБХІДНИХ СЕРВЕРНИХ РЕСУРСІВ.....11
3.1 Вихідні дані та розрахунок навантаження.....11
3.2 Потреба сервісів у ресурсах.....12
4 РОЗРОБКА КОНТЕЙНЕРІВ ТА АЛГОРИТМУ ЇХ ОРКЕСТРУВАННЯ.....14
4.1 Склад і вміст контейнерів.....14
4.2 Алгоритм оркестрування.....15
5 ДОСЛІДЖЕННЯ МОЖЛИВОСТЕЙ ОРГАНІЗАЦІЇ ХМАРНОГО СЕРВІСУ.....17
5.1 Моделі хмарних сервісів.....17
5.2 Порівняння хмарних пропозицій.....17
5.3 Гібридна схема edge – cloud.....18
6 ПРОПОЗИЦІЯ КЛАСТЕРНОЇ ОРГАНІЗАЦІЇ СЕРВЕРІВ.....19
6.1 Склад і топологія кластера.....19
6.2 Відмовостійкість кластера.....20
ВИСНОВКИ.....21
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....22

ВСТУП

В індивідуальному завданні 1 для теми № 4 кафедри КІЕ «Сервер „Розумного будинку“ на базі апаратної платформи Raspberry Pi з підтримкою інтеграції цифрового двійника» було спроєктовано базову інфраструктуру DevOps: репозиторій, конвеєр CI/CD у GitHub Actions, контейнери Docker Compose на одному Raspberry Pi 4, моніторинг і резервне копіювання. Сервер є центральним вузлом «Розумного будинку», навколо якого побудовано й інші теми кафедри [17].

Такої інфраструктури достатньо для одного стенда, але вона не відповідає сучасному стану IT-галузі за трьома ознаками. По-перше, сервер є єдиною точкою відмови: вихід з ладу плати зупиняє збір даних і керування кліматом. По-друге, Docker Compose не забезпечує автоматичного перезапуску сервісів на іншому вузлі та поступового оновлення без простою. По-третє, цифровий двійник і довгострокове зберігання даних потребують ресурсів, яких немає на Raspberry Pi, тож доцільно залучити хмару.

Мета роботи – запропонувати інфраструктуру DevOps, яка забезпечить відповідність теми сучасним підходам: розподіленій клієнт-серверній архітектурі, оркестрованим контейнерам, гібридній моделі edge – cloud і кластеризації серверів.

Для досягнення мети поставлено такі **задачі**:

1) розглянути модель клієнт-сервер і можливості її впровадження у проєкт;
2) проаналізувати переваги та недоліки багаторівневої моделі інфраструктури;
3) визначити необхідні серверні ресурси;
4) розробити склад контейнерів і алгоритм їх оркестрування;
5) дослідити можливості організації хмарного сервісу;
6) запропонувати кластерну організацію серверів.

1 МОДЕЛЬ КЛІЄНТ-СЕРВЕР ТА МОЖЛИВОСТІ ЇЇ ВПРОВАДЖЕННЯ У НАУКОВО-ТЕХНІЧНИЙ ПРОЄКТ

1.1 Сутність і різновиди моделі клієнт-сервер

Модель клієнт-сервер – це спосіб організації розподіленої системи, у якому сервер надає ресурси або послуги, а клієнти ініціюють запити до нього через мережу [12]. Сервер зосереджує дані й бізнес-логіку, тому їх легше захищати, резервувати та оновлювати, ніж у разі розподілення логіки між усіма пристроями.

Розрізняють кілька різновидів моделі:

– **запит – відповідь** (HTTP/REST): клієнт надсилає запит і чекає відповіді; підходить для веб-інтерфейсу та API;

– **публікація – підписка** (MQTT): клієнти обмінюються повідомленнями через посередника-брокера за темами; відправник не знає одержувачів, тому датчики й сервіси можна додавати незалежно;

– **двобічний потік** (WebSocket): після встановлення з’єднання сервер сам надсилає оновлення клієнтові; підходить для синхронізації стану з цифровим двійником у реальному часі.

За розподілом логіки виділяють «тонкі» клієнти, які лише відображають дані (браузер), і «товсті», що виконують частину обробки самостійно (датчик ESP8266 з локальною калібровкою).

1.2 Клієнт-серверна модель сервера «Розумного будинку»

Для проєкту модель клієнт-сервер є природною: Raspberry Pi виступає сервером, а датчики, контролер реле, браузер користувача та цифровий двійник – клієнтами (рисунок 1.1). Протоколи взаємодії наведено в таблиці 1.1.

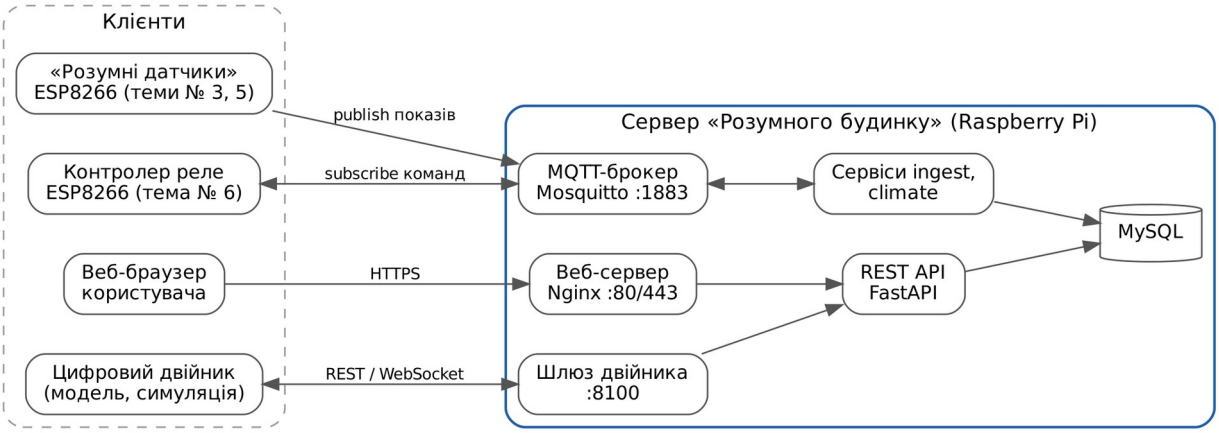


Рисунок 1.1 – Клієнт-серверна модель сервера «Розумного будинку»

Таблиця 1.1 – Клієнти сервера та протоколи взаємодії

Клієнт	Протокол, порт	Модель	Призначення
«Розумні датчики» ESP8266 (теми № 3, 5)	MQTT 3.1.1, 1883 (LAN)	публікація	Передавання показів температури, вологості, газів кожні 10 с
Контролер реле ESP8266 (тема № 6)	MQTT, 1883	підписка	Отримання команд увімкнення опалення та кондиціонування, звіт про стан
Веб-браузер користувача	HTTPS, 443	запит – відповідь	Сторінки налаштування датчиків і статистики
Цифровий двійник	REST + WebSocket, 443	двобічний потік	Отримання стану будинку, надсилання керувальних впливів, сценарії симуляції
Система моніторингу (Prometheus)	HTTP, 9100/9104	запит – відповідь	Збирання метрик вузлів і СКБД

Впровадження моделі у проєкт потребує трьох рішень. Перше – розділити ролі всередині сервера: брокер повідомлень, сервіси обробки, API та СКБД працюють окремими процесами (контейнерами), що дозволяє масштабувати й оновлювати їх незалежно. Друге – зафіксувати контракти: формат тем і повідомлень MQTT та специфікацію OpenAPI для REST, щоб клієнти й сервер розвивалися окремо. Третє – захистити канали: автентифікація клієнтів MQTT, TLS для зовнішніх з’єднань і доступ до сервера ззовні лише через зворотний проксі.

Недоліком класичної клієнт-серверної моделі є централізація: сервер стає єдиною точкою відмови та вузьким місцем. Для проєкту цей недолік усувається багаторівневою моделлю (розділ 2) і кластеризацією (розділ 6).

2 АНАЛІЗ ПЕРЕВАГ ТА НЕДОЛІКІВ ЗАСТОСУВАННЯ БАГАТОРІВНЕВОЇ МОДЕЛІ ІНФРАСТРУКТУРИ ДЛЯ НАУКОВО-ТЕХНІЧНОГО ПРОЄКТУ

2.1 Варіанти багаторівневої моделі

Багаторівнева (n-tier) модель розвиває клієнт-серверну, розділяючи систему на рівні з чіткими інтерфейсами між ними. Для проєкту розглянуто три варіанти (таблиця 2.1).

Таблиця 2.1 – Порівняння варіантів моделі інфраструктури

Критерій	**Дворівнева (І3-1)**	**Трирівнева**	**Чотирирівнева edge – fog – cloud**
Розташування рівнів	Пристрої + один Raspberry Pi	Представлення, застосунки, дані на кластері в лабораторії	Пристрої, edge-кластер, дані, хмара
Робота без Інтернету	так	так	так (керування на edge), хмарні функції недоступні
Масштабування	немає	горизонтальне в межах лабораторії	горизонтальне + хмарні ресурси за потреби
Ресурси для цифрового двійника	обмежені 2 ГБ ОЗП	обмежені вузлами кластера	достатні (хмарна VM)
Складність супроводу	низька	середня	висока
Вартість	мінімальна	обладнання кластера	обладнання + хмара (можливі безкоштовні тарифи)

Обрано чотирирівневу модель (рисунок 2.1). Рівень 1 – пристрої та клієнти представлення. Рівень 2 – застосунки на edge-кластері Raspberry Pi, які мають працювати навіть без Інтернету, бо керують опаленням. Рівень 3 – дані: MySQL, метрики, резервні копії. Рівень 4 – хмара для цифрового двійника, довгострокового архіву та віддалених дашбордів.

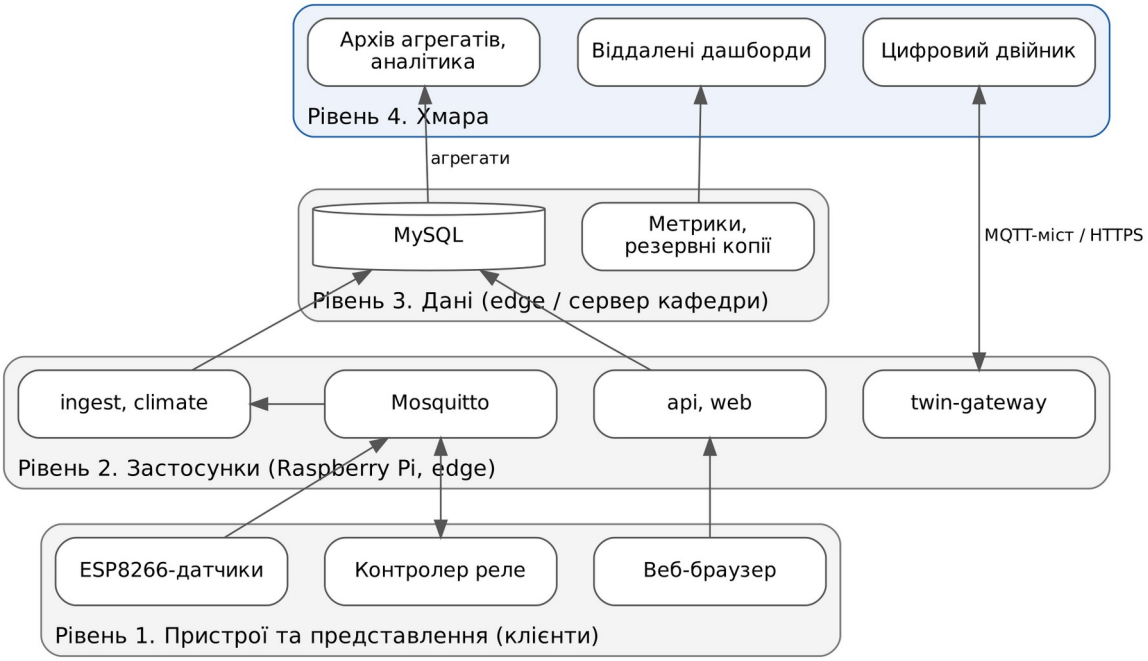


Рисунок 2.1 – Багаторівнева модель інфраструктури проєкту

2.2 Переваги та недоліки для науково-технічного проєкту

Результати аналізу зведено в таблицю 2.2.

Таблиця 2.2 – Переваги та недоліки багаторівневої моделі для проєкту

Переваги	Недоліки
Незалежне оновлення рівнів: нову версію моделі двійника можна розгорнути в хмарі, не зупиняючи збір даних на edge	Більше компонентів і мережевих з’єднань – більше точок можливої відмови та складніша діагностика
Керування кліматом лишається на edge з малою затримкою і працює без Інтернету	Затримка між edge і хмарою (десятки – сотні мілісекунд) не дозволяє переносити в хмару контур керування реле
Кожен рівень масштабується окремо: важкі обчислення двійника – у хмарі, легкий прийом даних – на Raspberry Pi	Потрібно узгоджувати версії інтерфейсів між рівнями (контракти MQTT і REST)
Безпека шарами: датчики бачать лише брокер, СКБД недоступна ззовні, у хмару передаються лише агреговані дані	Потрібен захищений канал edge – хмара (TLS, VPN) і керування сертифікатами
Модель відповідає еталонній архітектурі IoT ISO/IEC 30141 [16] і промисловій практиці, що підвищує цінність проєкту для навчання	Вищий поріг входження для нових учасників команди; потрібна документація та автоматизація (IaC)
Хмарні ресурси можна вмикати лише на час експериментів, що знижує вартість	Залежність від умов хмарного провайдера, зокрема зміни безкоштовних тарифів

Недоліки компенсуються засобами DevOps з І3-1: автоматичним розгортанням з коду, централізованим моніторингом і версіонованими контрактами. Тому переваги багаторівневої моделі для проєкту переважують.

3 ВИЗНАЧЕННЯ НЕОБХІДНИХ СЕРВЕРНИХ РЕСУРСІВ ДЛЯ ОБСЛУГОВУВАННЯ НАУКОВО-ТЕХНІЧНОГО ПРОЄКТУ

3.1 Вихідні дані та розрахунок навантаження

Навантаження оцінено для цільової конфігурації лабораторії: 20 «розумних датчиків», кожен передає 6 параметрів (температура, вологість, тиск, MQ-2, MQ-135, MQ-5) з періодом 10 с. Розмір повідомлення MQTT разом із заголовками – близько 300 Б, розмір рядка таблиці показів у MySQL з індексами – близько 100 Б.

Обсяг сирих даних за добу розраховано за формулою

$$V_{\text{доб}} = N \cdot m \cdot (86\,400 / T) \cdot s,$$

де N – кількість датчиків, m – кількість параметрів, T – період передавання, с, s – розмір рядка, Б. Результати розрахунку наведено в таблиці 3.1.

Таблиця 3.1 – Розрахунок навантаження та обсягу даних

Показник	Значення	Примітка
Потік повідомлень MQTT	2 повідомлення/с	20 датчиків / 10 с
Повідомлень за місяць	≈ 5,2 млн	важливо для тарифів хмарних брокерів
Вхідний трафік	≈ 4,8 кбіт/с; 1,6 ГБ/міс	для LAN 100–1000 Мбіт/с незначний
Рядків показів за добу	1 036 800	N · m · 86 400 / T
Сирі дані за добу	≈ 104 МБ	V доб при s = 100 Б
Сирі дані за рік без очищення	≈ 38 ГБ	не прийнятно для microSD і недоцільно для SSD плати
Сирі дані, зберігання 30 діб	≈ 3,1 ГБ	прийнята політика зберігання
Хвилинні агрегати за рік	≈ 7,6 ГБ	min / max / avg, рядок ≈ 120 Б; зберігаються безстроково, копія в хмарі

Висновок з розрахунку: обчислювальне навантаження від датчиків мале, а визначальним є обсяг даних. Тому потрібна політика зберігання (30 діб сирих

даних плюс агрегати) і SSD замість карти microSD. Розрахунок також показує, що 2 повідомлення/с не потребують окремого потужного брокера.

3.2 Потреба сервісів у ресурсах

Потребу кожного сервісу в процесорі та пам’яті задано запитами (requests) і лімітами (limits) Kubernetes на основі ІЗ-1 (таблиця 3.2). Окремо враховано ресурси самого оркестратора: за вимірюваннями розробників k3s, сервер k3s з робочим навантаженням потребує близько 1,6 ГБ ОЗП, на Raspberry Pi 4 – близько 30 % одного ядра; агент k3s – близько 275 МБ [1, 2].

Таблиця 3.2 – Ресурси сервісів проєкту

Сервіс	Реплік	CPU, запит	ОЗП, запит / ліміт	Сховище
mysql	1 + репліка	250m	768 / 1024 МіБ	40 ГіБ SSD
mosquitto	1	50m	32 / 64 МіБ	1 ГіБ
api	2	2 × 100m	2 × 128 / 192 МіБ	–
ingest	1	100m	96 / 128 МіБ	–
climate	1	50m	96 / 128 МіБ	–
twin-gateway	1	50m	96 / 128 МіБ	–
web	2	2 × 25m	2 × 32 / 64 МіБ	–
Разом (без оркестратора)		750m	1408 / 1984 МіБ	≈ 41 ГіБ
k3s server (на кожен з 3 вузлів)	3	≈ 0,3 ядра	≈ 1,6 ГіБ	etcd ≈ 1 ГіБ

Звідси випливає ключове обмеження: Raspberry Pi 4 з 2 ГБ ОЗП, обраний в ІЗ-1, не вміщує одночасно сервер k3s (≈ 1,6 ГБ) і сервіси проєкту (до 2 ГБ). Для оркестрованої інфраструктури потрібні вузли з 4–8 ГБ ОЗП. За трьох вузлів сумарно 20 ГБ ОЗП (8 + 8 + 4) після відрахування 4,8 ГБ на k3s лишається ≈ 15 ГБ – у 7,5 раза більше за ліміти сервісів. Цього запасу досить, щоб пережити відмову будь-якого вузла (розділ 6).

Для хмарного рівня потрібна VM ARM64 з 2 vCPU і 8–12 ГБ ОЗП: модель цифрового двійника, Grafana і архів агрегатів (≈ 8 ГБ/рік). Вимоги до мережі –

стабільне з’єднання з пропускною здатністю від 1 Мбіт/с, оскільки в хмару передаються лише агрегати й стан двійника.

4 РОЗРОБКА КОНТЕЙНЕРІВ, ЇХ ВМІСТУ ТА АЛГОРИТМУ ЇХ ОРКЕСТРУВАННЯ

4.1 Склад і вміст контейнерів

Програмне забезпечення сервера розділено на сім контейнерів, кожен з яких виконує одну функцію (таблиця 4.1). Образи власних сервісів збираються багатоетапно: на першому етапі встановлюються залежності, у фінальний образ потрапляють лише середовище виконання та код [5]. Це зменшує розмір образу й кількість потенційних вразливостей. Усі образи збираються для linux/arm64 конвеєром з І3-1, процеси в контейнерах працюють від непривілейованого користувача.

Таблиця 4.1 – Контейнери проєкту та їх вміст

Контейнер	Базовий образ	Вміст і функції	Порт	Перевірка стану
mysql	mysql:8.4	СКБД з показами, налаштуваннями датчиків, журналом команд; бінарний журнал для реплікації	3306	mysqladmin ping
mosquitto	eclipse-mosquitto:2	MQTT-брокер з автентифікацією, збереженням сесій і мостом до хмари	1883	TCP-порт 1883
ingest	python:3.12-slim	Підписка на теми датчиків, валідація, запис у БД, обчислення хвилинних агрегатів	8000	HTTP /health
api	python:3.12-slim	FastAPI: REST для веб-інтерфейсу, статистика, налаштування датчиків	8000	HTTP /health
climate	python:3.12-slim	Алгоритм керування опаленням і кондиціонуванням, команди реле через MQTT	8000	HTTP /health
twin-gateway	python:3.12-slim	Мікросервісні функції для цифрового двійника: REST і WebSocket, синхронізація стану	8000	HTTP /health
web	nginx-unprivileged:1.27	Статичні сторінки налаштування датчиків і статистики	8080	HTTP /

Конфігурація передається в контейнери через ConfigMap, а паролі – через Secret Kubernetes, тому один і той самий образ працює в усіх середовищах. Дані MySQL і Mosquitto зберігаються в постійних томах на SSD вузла. Тексти Dockerfile і маніфестів Kubernetes подано в електронних додатках А і Б.

4.2 Алгоритм оркестрування

Для оркестрування обрано k3s – сертифікований дистрибутив Kubernetes, оптимізований для ARM і пристроїв з обмеженими ресурсами [1]. Порівняно з Docker Compose він автоматично перезапускає контейнери на іншому вузлі, оновлює сервіси без простою та розподіляє навантаження. Оркестрування виконується за таким алгоритмом:

1) **Розміщення.** Планувальник k3s розміщує поди з урахуванням запитів ресурсів (таблиця 3.2) і правил: MySQL primary і Mosquitto – на вузлі з міткою db-primary, репліки api розносяться по різних вузлах (topologySpreadConstraints), на хмарний вузол потрапляють лише поди з явним дозволом (taint).

2) **Порядок запуску.** Спочатку запускаються MySQL і Mosquitto, далі – ingest, api, climate, twin-gateway, web. Залежні сервіси не отримують трафік, доки їх readinessProbe не підтвердить з’єднання з БД і брокером [4].

3) **Самовідновлення.** livenessProbe перевіряє кожен контейнер кожні 15–20 с; після трьох невдалих перевірок контейнер перезапускається. Якщо вузол недоступний понад 5 хв, його поди переносяться на інші вузли.

4) **Оновлення.** Після публікації нового тегу образу конвеєр CI (І3-1) запускає оновлення за алгоритмом рисунка 4.1: резервна копія БД, міграції, поступова заміна подів (maxSurge = 1, maxUnavailable = 0), перевірка готовності, автоматичний відкат у разі невдачі [3].

5) **Обслуговування.** CronJob щодоби о 02:30 створює резервну копію MySQL на сервер кафедри; застарілі копії (понад 14 діб) видаляються.

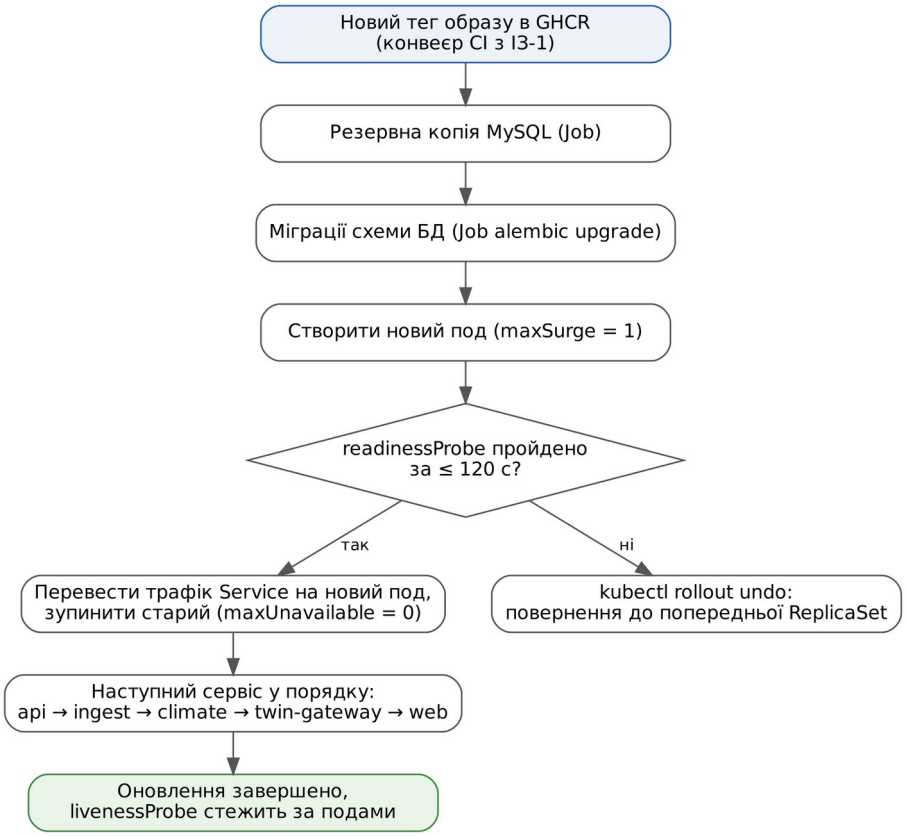


Рисунок 4.1 – Алгоритм оновлення контейнерів в оркестраторі k3s

Сервіс climate оновлюється стратегією Recreate, а не поступовою заміною: два екземпляри алгоритму керування не повинні одночасно надсилати команди реле. Його короткий простій (до 30 с) безпечний, бо реле зберігають останній стан.

5 ДОСЛІДЖЕННЯ МОЖЛИВОСТЕЙ ОРГАНІЗАЦІЇ ХМАРНОГО СЕРВІСУ ДЛЯ ОБСЛУГОВУВАННЯ НАУКОВО-ТЕХНІЧНОГО ПРОЄКТУ

5.1 Моделі хмарних сервісів

За класифікацією NIST виділяють три моделі хмарних сервісів: IaaS (інфраструктура – віртуальні машини, мережі, сховища), PaaS (платформа – керовані СКБД, брокери, середовища виконання) і SaaS (готові застосунки) [11]. Для IoT-проєктів додатково існують спеціалізовані керовані сервіси: MQTT-брокери та платформи цифрових двійників.

Для проєкту хмара потрібна для трьох функцій: обчислення моделі цифрового двійника, довгострокове зберігання агрегованих даних і віддалений доступ до дашбордів. Контур керування опаленням у хмару не переноситься через затримки та залежність від Інтернету.

5.2 Порівняння хмарних пропозицій

Порівняно пропозиції, доступні студентові без значних витрат (таблиця 5.1). Ціни та ліміти безкоштовних тарифів змінюються, тому значення наведено станом на листопад 2026 р. за даними провайдерів.

Таблиця 5.1 – Порівняння хмарних сервісів для проєкту

Пропозиція	Модель	Що надає і скільки коштує	Придатність для проєкту
Oracle Cloud Always Free (Ampere A1)	IaaS	VM ARM64 сумарно 2 OCPU і 12 ГБ ОЗП, 200 ГБ блочного сховища безкоштовно; у 2026 р. ліміт зменшено вдвічі з 4 OCPU / 24 ГБ; потрібна картка для верифікації [6, 7]	Висока: архітектура ARM64 збігається з Raspberry Pi, ресурсів досить для двійника й Grafana; ризик – зміни умов і видалення неактивних акаунтів
Azure for Students	IaaS / PaaS	100 дол. США кредиту на 12 місяців без кредитної картки, поновлюється щороку для студентів [8]	Висока для експериментів: можна спробувати керовані сервіси (IoT Hub, контейнери); кредиту недостатньо для постійної роботи потужної VM

Пропозиція	Модель	Що надає і скільки коштує	Придатність для проєкту
Hetzner Cloud CAX11	IaaS	VM ARM64: 2 vCPU, 4 ГБ, 40 ГБ; після підвищення цін у 2026 р. ≈ 6,5 євро/міс разом з IPv4 [9, 10]	Середня: передбачувана платна альтернатива, якщо безкоштовні тарифи зникнуть
HiveMQ Cloud Serverless	PaaS (MQTT)	Безкоштовно: 100 з’єднань і 10 ГБ трафіку на місяць, MQTT 5.0 з TLS [13]	Висока як хмарна точка мосту: потік ≈ 1,6 ГБ/міс вміщується в ліміт
AWS IoT Core	PaaS (IoT)	1 дол. США за 1 млн повідомлень і 0,08 дол. США за 1 млн хвилин з’єднання; 5,2 млн повідомлень/міс ≈ 5,3 дол. США/міс [14]	Середня: зручна інтеграція з сервісами AWS, але платна для постійного потоку від усіх датчиків

5.3 Гібридна схема edge – cloud

За результатами порівняння запропоновано гібридну схему: усі сервіси реального часу лишаються на edge-кластері в лабораторії, а хмара використовується як додатковий рівень (див. рисунок 2.1):

– **хмарний вузол** – VM Oracle Cloud Always Free (ARM64, 2 OCPU, 12 ГБ) приєднується до кластера k3s як агент через тунель WireGuard; на ньому працюють модель цифрового двійника, Grafana та архів агрегатів;

– **обмін даними** – міст Mosquitto передає в хмару лише агреговані покази та стан двійника з якістю обслуговування QoS 1 і буферизацією на час розриву зв’язку [15]; сирі дані з лабораторії не виходять;

– **резерв** – у разі зміни умов Oracle той самий набір маніфестів розгортається на Hetzner CAX11 або в Azure за кредитом для студентів, бо всі сервіси описано як код;

– **безпека** – доступ до хмарного вузла лише через WireGuard, зовнішні запити – через Ingress з TLS, секрети зберігаються в Secret Kubernetes.

Така схема зберігає роботу «Розумного будинку» без Інтернету, а витрати на хмару для навчального проєкту за безкоштовних тарифів дорівнюють нулю.

6 ПРОПОЗИЦІЯ КЛАСТЕРНОЇ ОРГАНІЗАЦІЇ СЕРВЕРІВ ДЛЯ РЕАЛІЗАЦІЇ НАУКОВО-ТЕХНІЧНОГО ПРОЄКТУ

6.1 Склад і топологія кластера

Запропоновано кластер k3s з трьох server-вузлів Raspberry Pi у лабораторії та одного агента в хмарі (рисунок 6.1, таблиця 6.1). Три server-вузли з вбудованою СКБД etcd утворюють відмовостійку площину керування: etcd зберігає працездатність, доки доступна більшість (2 з 3) вузлів. Розробники k3s рекомендують для вузлів на Raspberry Pi зовнішній SSD, бо etcd інтенсивно записує на диск, а карти microSD такого навантаження не витримують [1].

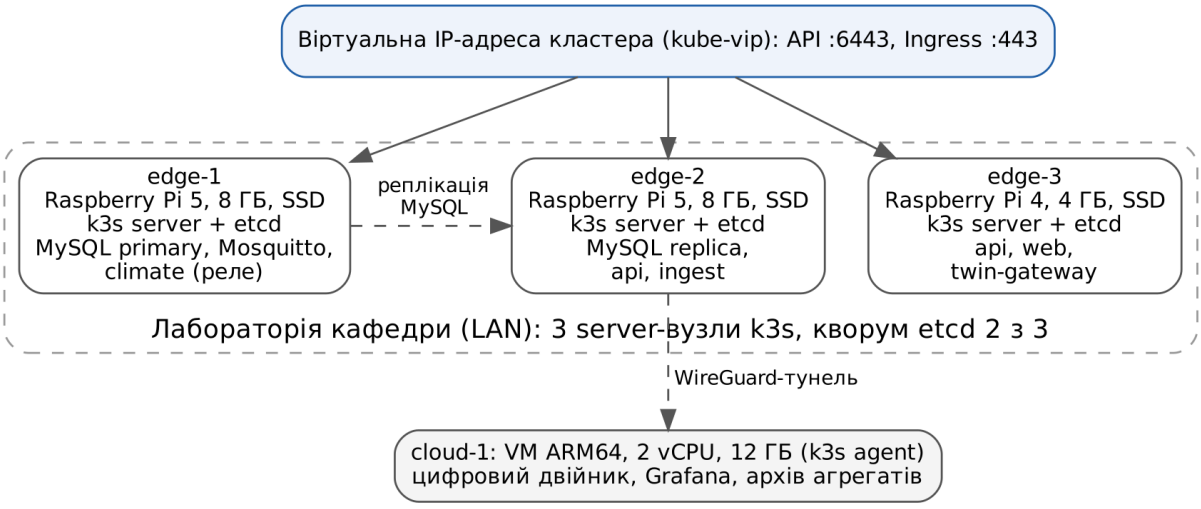


Рисунок 6.1 – Топологія кластера серверів

Таблиця 6.1 – Склад кластера

Вузол	Апаратне забезпечення	Роль k3s	Основні поди
edge-1	Raspberry Pi 5, 8 ГБ, SSD 128 ГБ	server + etcd	MySQL primary, Mosquitto, climate, api
edge-2	Raspberry Pi 5, 8 ГБ, SSD 128 ГБ	server + etcd	MySQL replica, api, ingest, web
edge-3	Raspberry Pi 4, 4 ГБ, SSD 64 ГБ (наявна плата з І3-1 після заміни на 4 ГБ)	server + etcd	twin-gateway, web, резерв для подів інших вузлів
cloud-1	VM ARM64 Oracle Cloud, 2 OCPU, 12 ГБ	agent (taint cloud)	Цифровий двійник, Grafana, архів агрегатів

Мережа: вузли edge з’єднані комутатором 1 Гбіт/с; віртуальна адреса kube-vip надає єдину точку входу для API кластера та Ingress, яка переходить на інший

вузол у разі відмови. Хмарний вузол підключено через WireGuard. Для захисту від відключень електроенергії вузли edge та комутатор живляться від ДБЖ.

6.2 Відмовостійкість кластера

Поведінку кластера в типових сценаріях відмов наведено в таблиці 6.2.

Таблиця 6.2 – Поведінка кластера при відмовах

Сценарій	Реакція кластера	Вплив на користувачів
Відмова edge-3	Кворум etcd 2 з 3 зберігається; поди twin-gateway і web переносяться на edge-1, edge-2	Короткочасна (до 5 хв) недоступність синхронізації з двійником
Відмова edge-2	Кворум зберігається; api та ingest працюють на інших вузлах; MySQL primary продовжує роботу без репліки	Без простою; знижується захищеність даних до відновлення репліки
Відмова edge-1 (MySQL primary)	Кворум зберігається; Mosquitto і climate перезапускаються на edge-2; адміністратор переводить репліку MySQL у режим primary за інструкцією	Прийом даних і керування призупиняються на 5–15 хв; дані до моменту відмови збережено в репліці
Втрата Інтернету	Міст MQTT буферизує повідомлення; хмарний агент позначається недоступним	Локальне керування й веб-інтерфейс працюють; двійник і віддалені дашборди недоступні
Відмова двох edge-вузлів	Кворум etcd втрачено, API кластера недоступне; запущені контейнери на вузлі, що лишився, продовжують працювати	Потрібне ручне відновлення з резервної копії etcd і БД

Отже, запропонований кластер витримує відмову будь-якого одного вузла, що усуває головний недолік конфігурації з І3-1 – єдину точку відмови. Скрипт розгортання кластера подано в електронному додатку В.

ВИСНОВКИ

У звіті запропоновано інфраструктуру DevOps, яка приводить сервер «Розумного будинку» на Raspberry Pi (тема № 4 кафедри КІЕ) у відповідність до сучасного стану ІТ-галузі.

1) Модель клієнт-сервер застосовано з трьома видами взаємодії: публікація – підписка MQTT для датчиків і реле, запит – відповідь HTTPS для веб-інтерфейсу, двобічний потік WebSocket для цифрового двійника; для клієнтів зафіксовано контракти та захист каналів.

2) Обрано чотирирівневу модель edge – fog – cloud: керування кліматом лишається на edge і працює без Інтернету, а цифровий двійник і архів винесено в хмару; недоліки моделі (складність, більше точок відмови) компенсуються засобами DevOps.

3) Розрахунок для 20 датчиків показав малий потік (2 повідомлення/с), але значний обсяг даних (≈ 104 МБ/добу, ≈ 38 ГБ/рік), що вимагає політики зберігання та SSD; Raspberry Pi 4 з 2 ГБ ОЗП недостатньо для оркестратора, потрібні вузли з 4–8 ГБ.

4) Розроблено сім контейнерів з багатоетапною збіркою і перевірками стану та алгоритм оркестрування в k3s з поступовим оновленням без простою й автоматичним відкатом.

5) Порівняно п’ять хмарних пропозицій; запропоновано гібридну схему з безкоштовною VM Oracle Cloud ARM64 і мостом MQTT, з резервними варіантами Hetzner і Azure for Students.

6) Запропоновано кластер з трьох server-вузлів k3s на Raspberry Pi з SSD і хмарного агента, який витримує відмову будь-якого одного вузла.

Електронні додатки до звіту: Dockerfile образів (додаток А), маніфести Kubernetes (додаток Б) і скрипт розгортання кластера k3s (додаток В).

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Requirements. *K3s Documentation*. URL: https://docs.k3s.io/installation/requirements (дата звернення: 06.11.2026).
2. Resource Profiling. *K3s Documentation*. URL: https://docs.k3s.io/reference/resource-profiling (дата звернення: 06.11.2026).
3. Deployments. *Kubernetes Documentation*. URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ (дата звернення: 06.11.2026).
4. Liveness, Readiness, and Startup Probes. *Kubernetes Documentation*. URL: https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/ (дата звернення: 06.11.2026).
5. Multi-stage builds. *Docker Docs*. URL: https://docs.docker.com/build/building/multi-stage/ (дата звернення: 06.11.2026).
6. Always Free Resources. *Oracle Cloud Infrastructure Documentation*. URL: https://docs.oracle.com/en-us/iaas/Content/FreeTier/resourceref.htm (дата звернення: 06.11.2026).
7. Oracle Quietly Halves Free Tier Ampere A1 Compute Limits with No Public Announcement. *InfoQ*. 2026. URL: https://infoq.com/news/2026/07/oracle-cloud-free-tier-limits/ (дата звернення: 06.11.2026).
8. Azure for Students. *Microsoft Azure*. URL: https://azure.microsoft.com/en-us/free/students (дата звернення: 06.11.2026).
9. Cost-optimized cloud servers. *Hetzner*. URL: https://www.hetzner.com/cloud/cost-optimized (дата звернення: 06.11.2026).
10. Hetzner Cloud Pricing 2026: Price Hikes & CX Availability. *Bitdoze*. URL: https://www.bitdoze.com/hetzner-cloud-cost-optimized-plans (дата звернення: 06.11.2026).
11. Mell P., Grance T. The NIST Definition of Cloud Computing : NIST Special Publication 800-145. Gaithersburg : NIST, 2011. 7 p.

12. Tanenbaum A. S., van Steen M. Distributed Systems. 3rd ed. Maarten van Steen, 2017. 596 p.

13. HiveMQ pricing. *HiveMQ*. URL: https://www.hivemq.com/hivemq/editions/ (дата звернення: 06.11.2026).

14. AWS IoT Core pricing. *Amazon Web Services*. URL: https://aws.amazon.com/iot-core/pricing/ (дата звернення: 06.11.2026).

15. mosquitto.conf – the configuration file for mosquitto. *Eclipse Mosquitto*. URL: https://mosquitto.org/man/mosquitto-conf-5.html (дата звернення: 06.11.2026).

16. ISO/IEC 30141:2018. Internet of Things (IoT) – Reference Architecture. Geneva : ISO, 2018.

17. Мардєєва Д. Р., Вадурін К. О., Перекрест І. А. Розробка структури Розумного будинку з використанням технології Industry 5.0. *Актуальні проблеми життєдіяльності суспільства* : матеріали XXX Міжнар. наук.-практ. конф. студентів, аспірантів та молодих учених КрНУ ім. М. Остроградського, 20–21 квітня 2023 р. Кременчук, 2023. С. 71–72.